



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Kevin Wangui
July 8, 2026



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Summary of methodologies
 - Data Collection
 - Data Wrangling
 - EDA with Data Visualization
 - EDA with SQL
 - Building an Interactive map with Folium
 - Building a Dashboard with Plotly Dash
 - Predictive Analysis (Classification)
- Summary of all results
 - EDA Results
 - Interactive Analytics
 - Predictive Analysis

Introduction

Project Background

SpaceX has transformed aerospace logistics by perfecting first-stage booster reuse. Standard competitors price launches up to \$165M, whereas a reusable Falcon 9 is offered at just \$62M. Predicting successful landings directly decodes launch economics.

Core Problems to Solve

How do payload mass, destination orbit, and launch site characteristics impact booster landing likelihood? Can machine learning predict return-to-Earth outcomes reliably to inform alternative commercial bidding strategies?



Section 1

Methodology

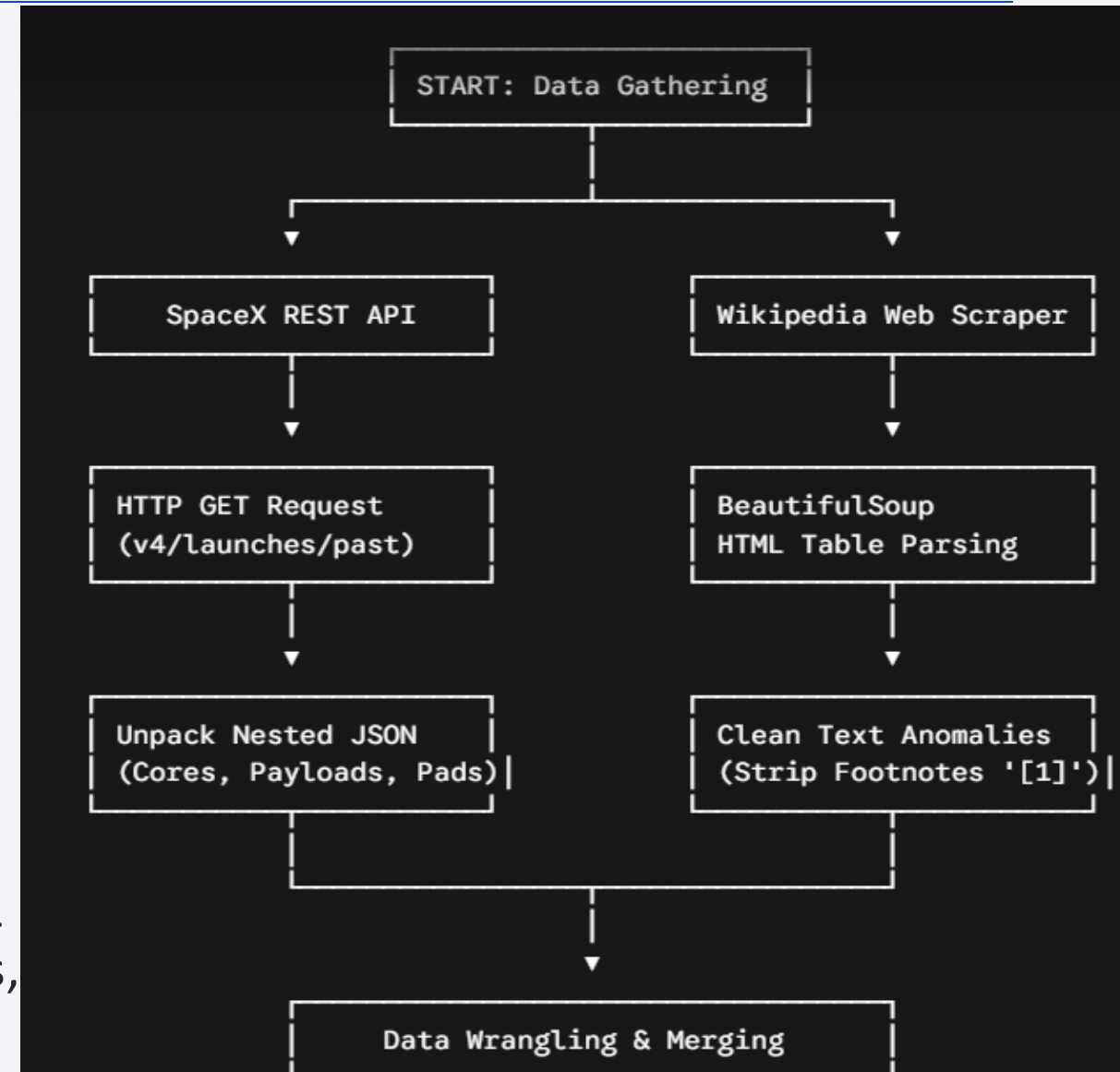
Methodology

Executive Summary

- Data collection methodology:
 - SpaceX REST API
 - Web scrapping from Wikipedia
- Perform data wrangling
 - Transformed categorical data using One Hot Encoding
 - Removing null values
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
 - Machine Learning models such KNN were built.

Data Collection

- **SpaceX REST API Integration:** Requested primary launch data from the official public SpaceX API endpoints to retrieve foundational technical data.
- Extracted core variables including launch specifications, booster versions, payload masses, core details, flight numbers, and launch site coordinates.
- **Wikipedia Web Scraping:**
- Scraped the "List of Falcon 9 and Falcon Heavy launches" Wikipedia page to supplement missing historical records and verified data consistency.
- Programmatically parsed unstructured HTML tables to extract third-generation launch logs, payload details, and landing outcomes.



Data Collection – SpaceX API

1. Request the SpaceX launch data using the GET request.

```
spacex_url="https://api.spacexdata.com/v4/launches/past"

response = requests.get(spacex_url)
```

2. Convert the JSON result into a DataFrame.

```
•[11]: # Use json_normalize method to convert the json result into a dataframe
data = pd.json_normalize(response.json())
```

3. Filter the DataFrame to only include Falcon 9 launches.

```
[24]: # Hint data['BoosterVersion']!='Falcon 1'
data_falcon9 = launch[launch["BoosterVersion"] == "Falcon 9"]
```

4. Dealing with Missing Values.

```
[27]: # Calculate the mean value of PayloadMass column
pm_mean = data_falcon9["PayloadMass"].mean()
nulls = data_falcon9[data_falcon9["PayloadMass"].isnull()]
print(f"Nulls in PayloadMass (before): {len(nulls)}")

# Replace the np.nan values with its mean value
# data_falcon9.replace(np.nan, pm_mean)
# data_falcon9["PayloadMass"].replace(np.nan, pm_mean)
func = lambda x: pm_mean if np.isnan(x) else x
data_falcon9.loc[:, "PayloadMass"] = data_falcon9["PayloadMass"].apply(func)

nulls = data_falcon9[data_falcon9["PayloadMass"].isnull()]
print(f"Nulls in PayloadMass (after): {len(nulls)}")
data_falcon9.isnull().sum()
```

5. Exporting the data it to a CSV file.

```
data_falcon9.to_csv('dataset_part_1.csv', index=False)
```

[GitHub Link](#)

Data Collection - Scraping

1. Request the Falcon9 Launch Wiki page from its URL and create a beautiful soup object.

```
[7]: response = requests.get(static_url, headers = headers)
```

Create a BeautifulSoup object from the HTML response

```
[9]: soup = BeautifulSoup(response.text, "html.parser")
```

3. Create a data frame by parsing the launch HTML tables.

```
[15]: launch_dict = dict.fromkeys(column_names)

# Remove an irrelevant column
del launch_dict['Date and time ( )']

# Let's initial the launch_dict with each value to be an empty list
launch_dict['Flight No.'] = []
launch_dict['Launch site'] = []
launch_dict['Payload'] = []
launch_dict['Payload mass'] = []
launch_dict['Orbit'] = []
launch_dict['Customer'] = []
launch_dict['Launch outcome'] = []
# Added some new columns
launch_dict['Version Booster'] = []
launch_dict['Booster landing'] = []
launch_dict['Date'] = []
launch_dict['Time'] = []
```

2. Extract all column/variable names from the HTML table header.

```
•[11]: html_tables = soup.find_all('table')
```

Starting from the third table is our target table contains the actual launch records.

```
•[12]: first_launch_table = html_tables[2]
print(first_launch_table)
```

```
<table class="wikitable plainrowheaders collapsible" style="width: 100%;">...
```

You should be able to see the columns names embedded in the table header elements `<th>` as follows:

```
''' ...
```

Next, we just need to iterate through the `<th>` elements and apply the provided `extract_column_from_header()` to extract column

```
•[13]: column_names = []

for th in first_launch_table.find_all('th'):
    name = extract_column_from_header(th)
    if name is not None and len(name) > 0:
        column_names.append(name)
```

Check the extracted column names

```
[14]: print(column_names)
```

```
['Flight No.', 'Date and time ( )', 'Launch site', 'Payload', 'Payload mass', 'Orbit', 'Customer', 'Launch outcome']
```

Data Wrangling

1. Calculated the number of launches on each site.

```
[6]: # Apply value_counts() on column LaunchSite
df['LaunchSite'].value_counts()
```

```
[6]: LaunchSite
      CCAFS SLC 40    55
      KSC LC 39A    22
      VAFB SLC 4E    13
      Name: count, dtype: int64
```

Each launch aims to an dedicated orbit, and here are some common orbit types:

2. Calculated the number and occurrence of each orbit.

```
[14]: # Apply value_counts on Orbit column
df['Orbit'][df['Orbit']!='GTO'].value_counts()
```

```
[14]: Orbit
      ISS    21
      VLEO   14
      PO     9
      LEO     7
      SSO     5
      MEO     3
      HEO     1
      ES-L1   1
      SO      1
      GEO     1
      Name: count, dtype: int64
```

3. Calculated the number and occurrence of mission outcome of the orbits.

```
[15]: # Landing_outcomes = values on Outcome column
landing_outcomes = df['Outcome'].value_counts()
landing_outcomes
```

```
[15]: Outcome
      True ASDS    41
      None None    19
      True RTLS    14
      False ASDS    6
      True Ocean    5
      False Ocean    2
      None ASDS     2
      False RTLS     1
      Name: count, dtype: int64
```

4. Created a landing outcome label from Outcome column.

```
[18]: # Landing_class = 0 if bad_outcome
      # Landing_class = 1 otherwise
      landing_class = [0 if outcome in bad_outcomes else 1 for outcome in df['Outcome']]
```

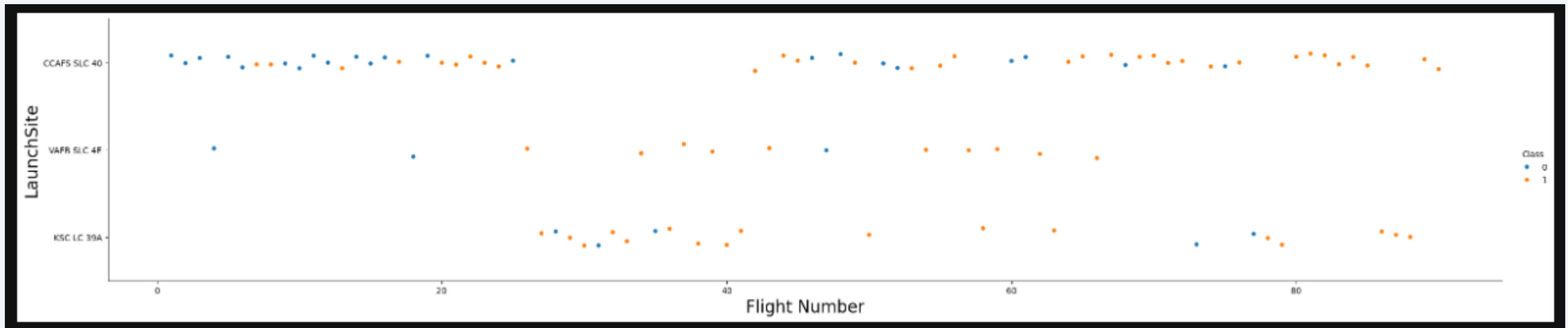
This variable will represent the classification variable that represents the outcome of each launch. If the launch did not land successfully, one means the first stage landed Successfully

```
[19]: df['Class']=landing_class
```

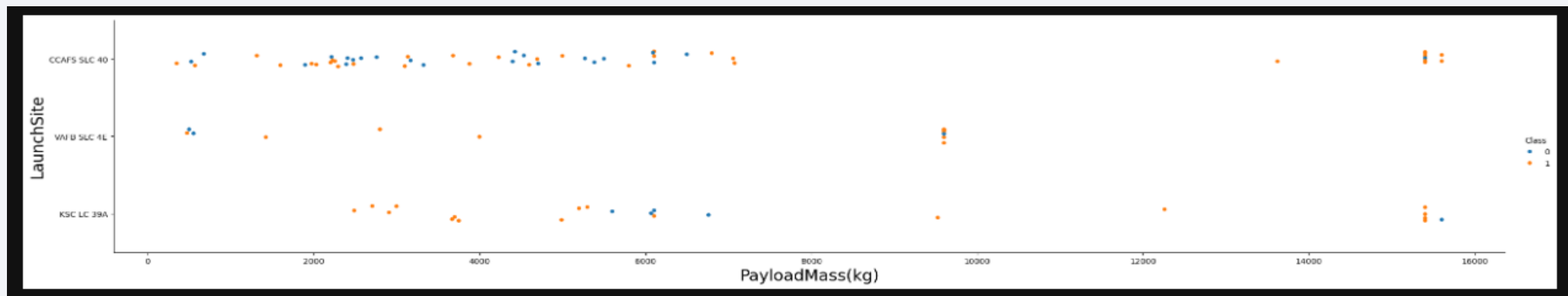
[GitHub Link](#)

EDA with Data Visualization

1. The relationship between Flight Number and Launch Site – Scatter Plot.

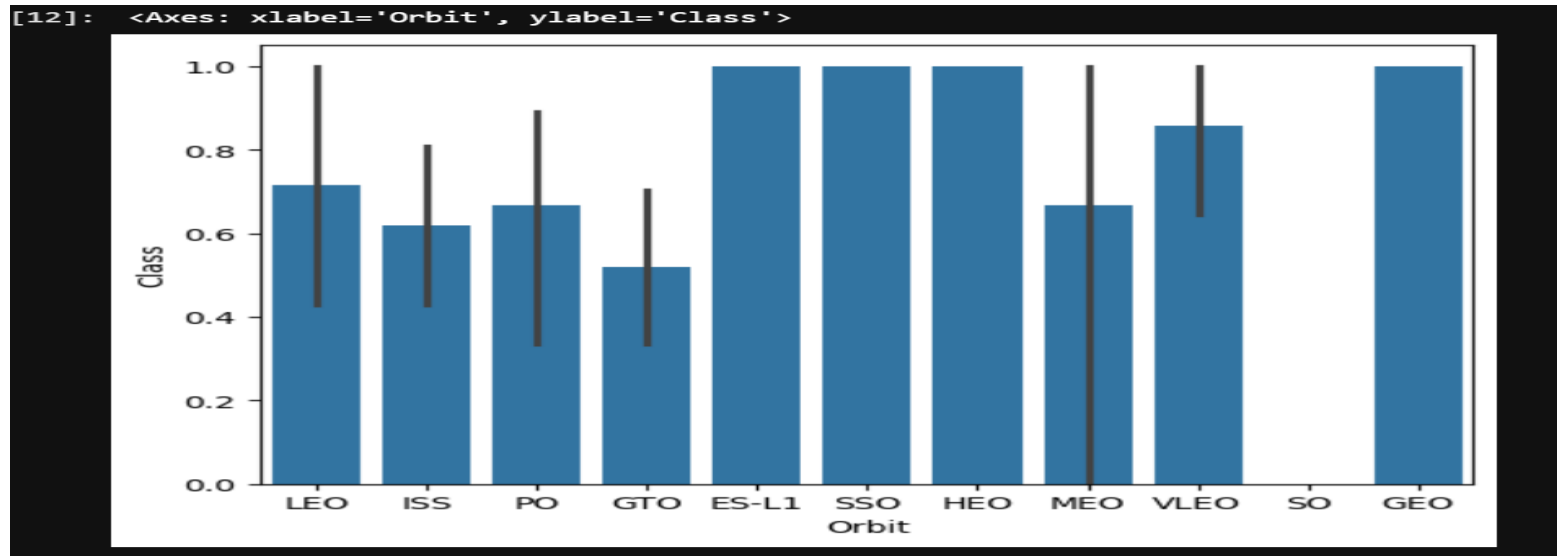


2. The relationship between Payload and Launch Site - Scatter Plot.

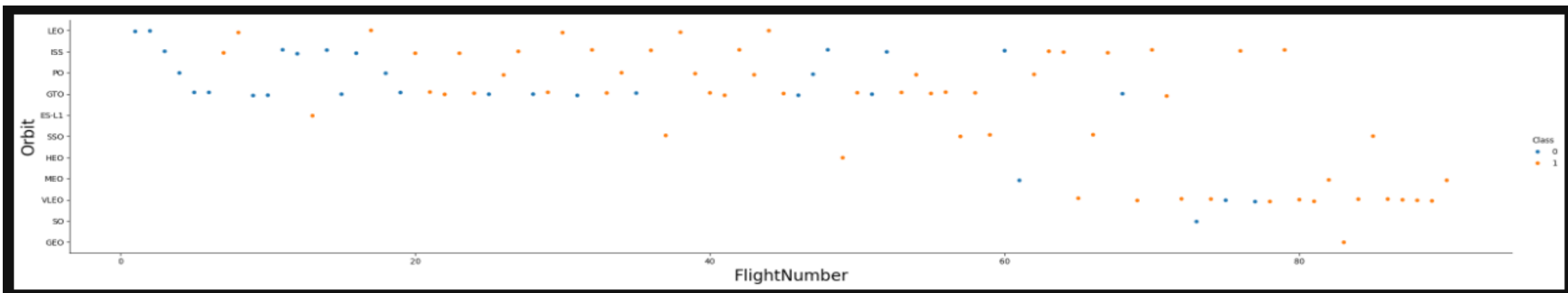


EDA with Data Visualization – Part 2

3. Bar chart for the success rate of each orbit.

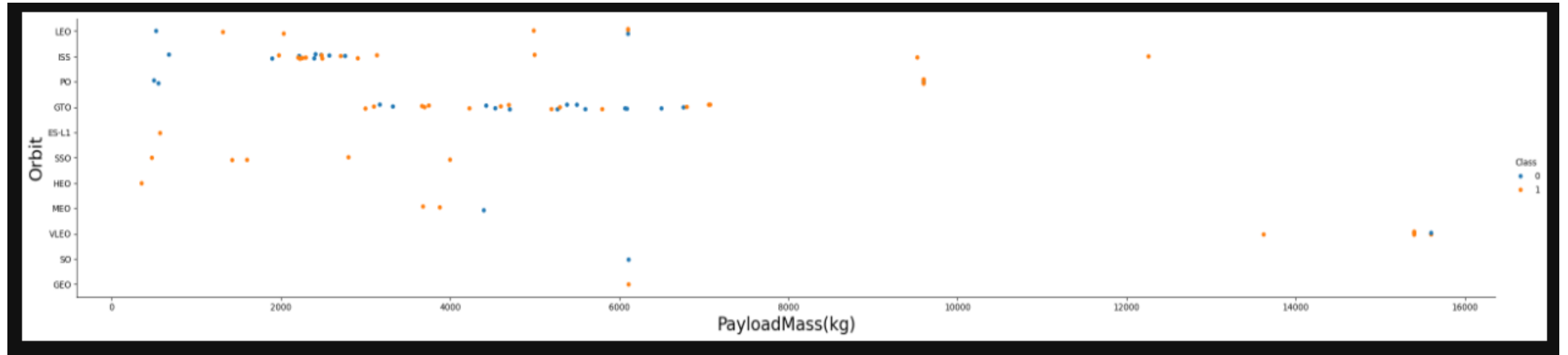


4. The relationship between Flight Number and Orbit type – Scatter Plot



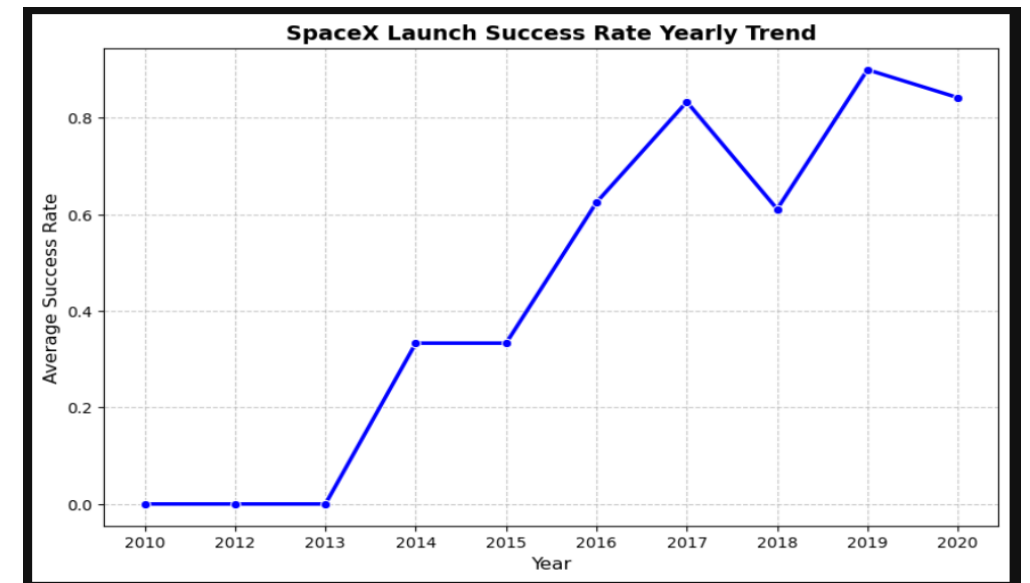
EDA with Data Visualization – Part 3

5. The relationship between Payload and Orbit type.



6. SpaceX the launch success yearly trend - Line Chart

[GitHub Link](#)



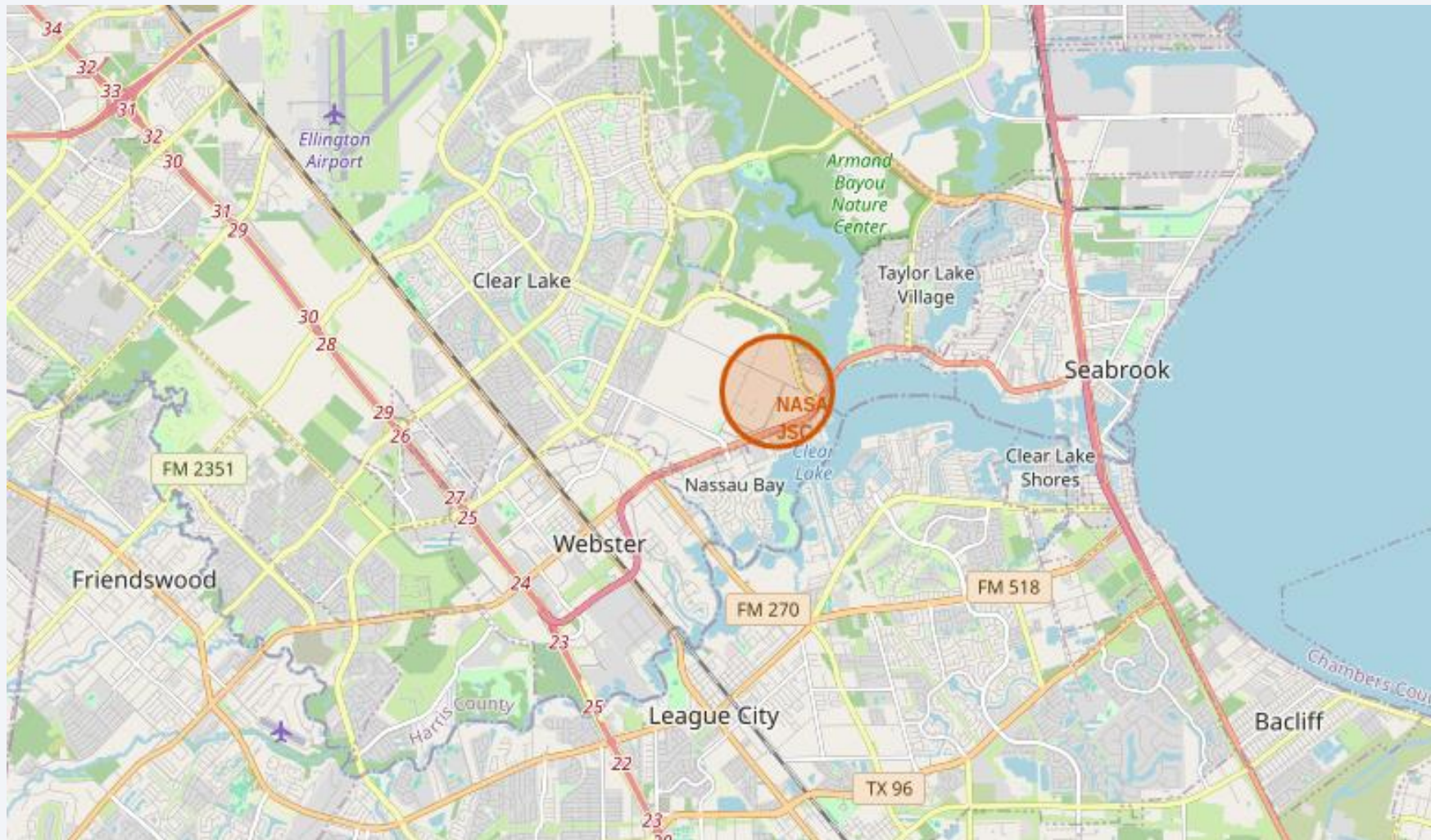
EDA with SQL

SQL Queries performed: - [GitHub Link](#)

- Displayed the names of the unique launch sites in the space mission.
- Displayed 5 records where launch sites begin with the string 'CCA'
- Displayed the total payload mass carried by boosters launched by NASA (CRS)
- Displayed average payload mass carried by booster version F9 v1.1
- Listed the date when the first successful landing outcome in ground pad was achieved.
- List the names of the boosters which have success in drone ship and have payload mass greater than 4000 but less than 6000.
- List the total number of successful and failure mission outcomes
- List all the booster_versions that have carried the maximum payload mass, using a subquery with a suitable aggregate function.
- List the records which will display the month names, failure landing_outcomes in drone ship ,booster versions, launch_site for the months in year 2015.
- Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order.

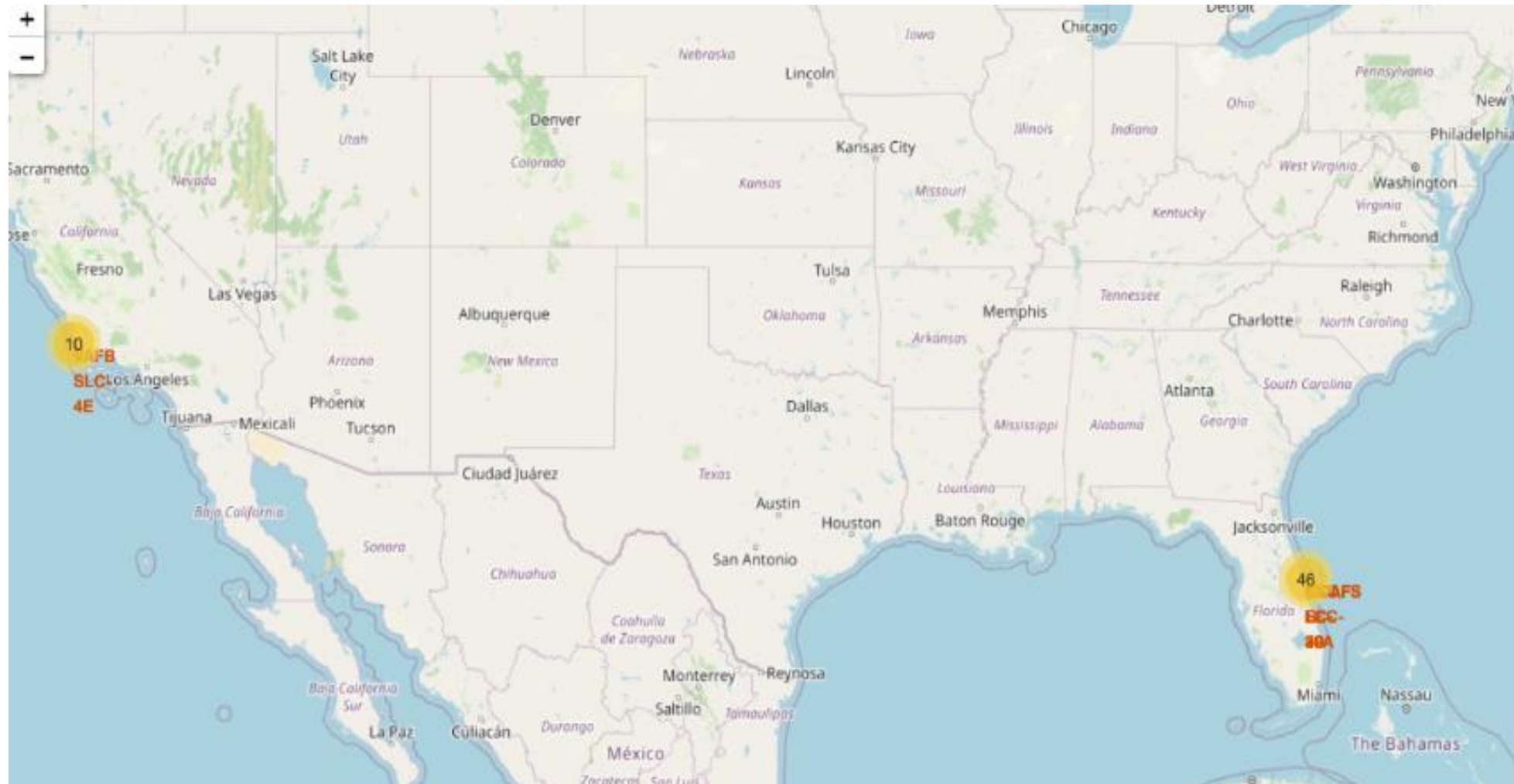
Build an Interactive Map with Folium

Circle Marker created to show NASA Johnson Space Center – [GitHub Link](#)



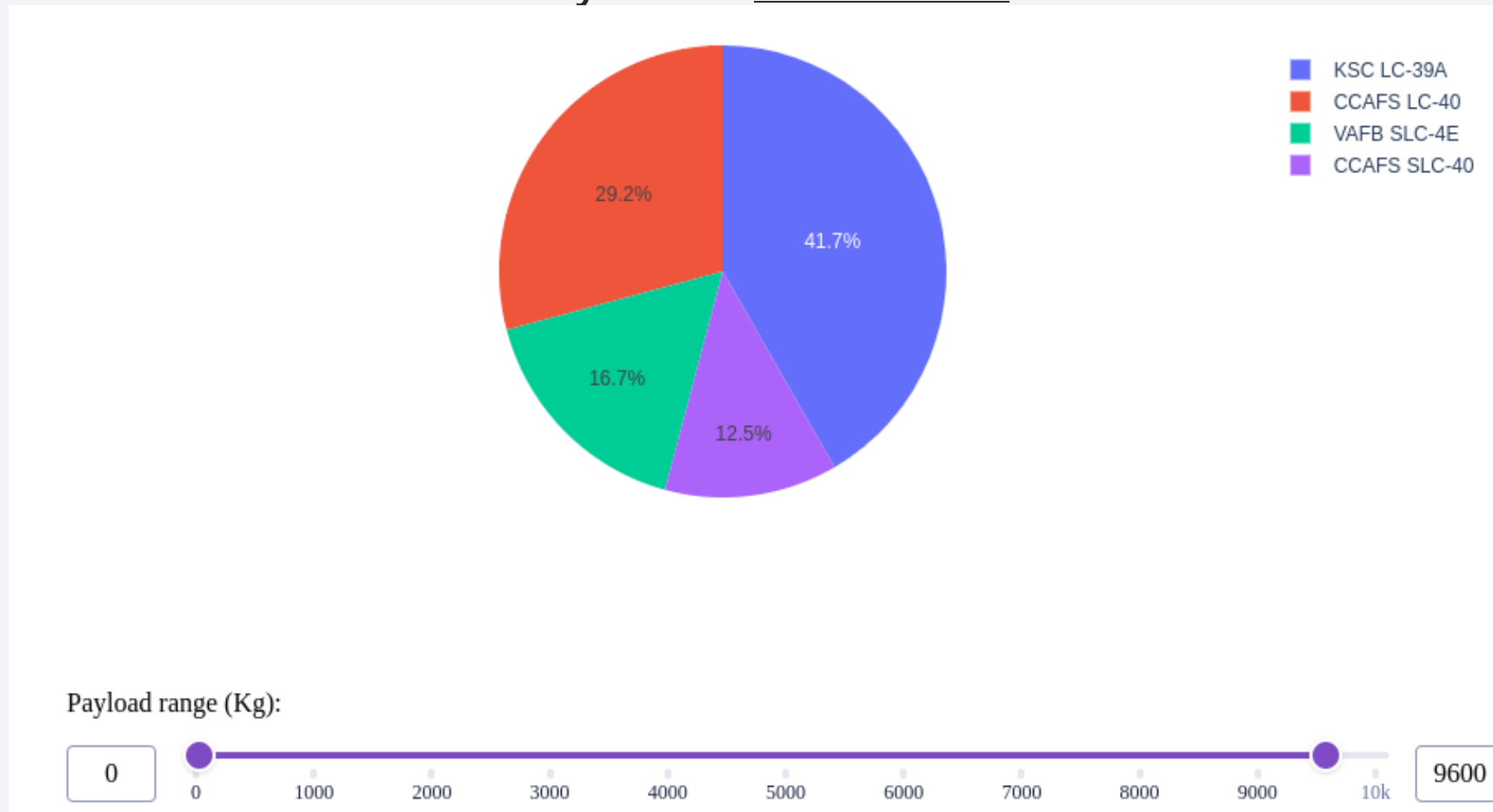
Build an Interactive Map with Folium – Part 2

Map Markers added for finding optimal launch sites. - [GitHub Link](#)



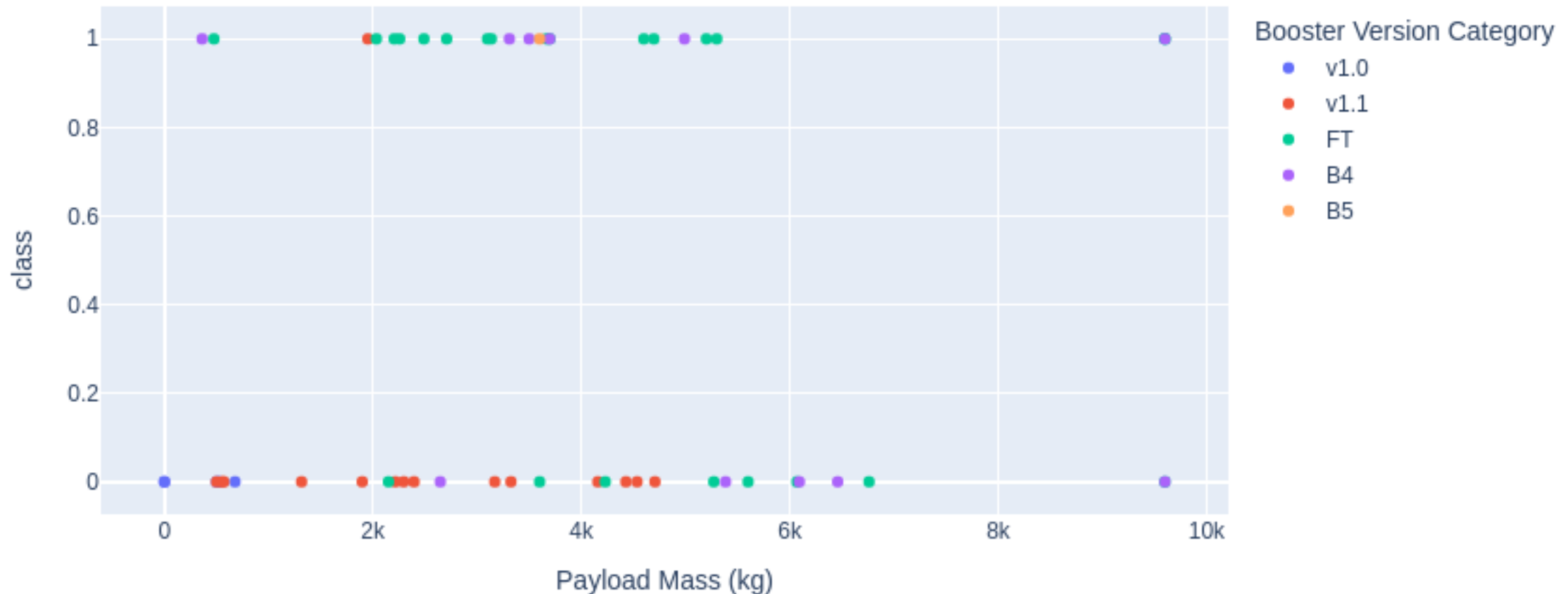
Build a Dashboard with Plotly Dash

Total Success Launches by Site. - [GitHub Link](#)



Build a Dashboard with Plotly Dash – Part 2

Correlation between Payload and Success for all Sites. - [GitHub Link](#)



Predictive Analysis (Classification)

1. Building the Pipeline (Data Preparation)
2. Hyperparameter Improvement (Tuning & Optimization)
3. Evaluation & Finding the Best Model

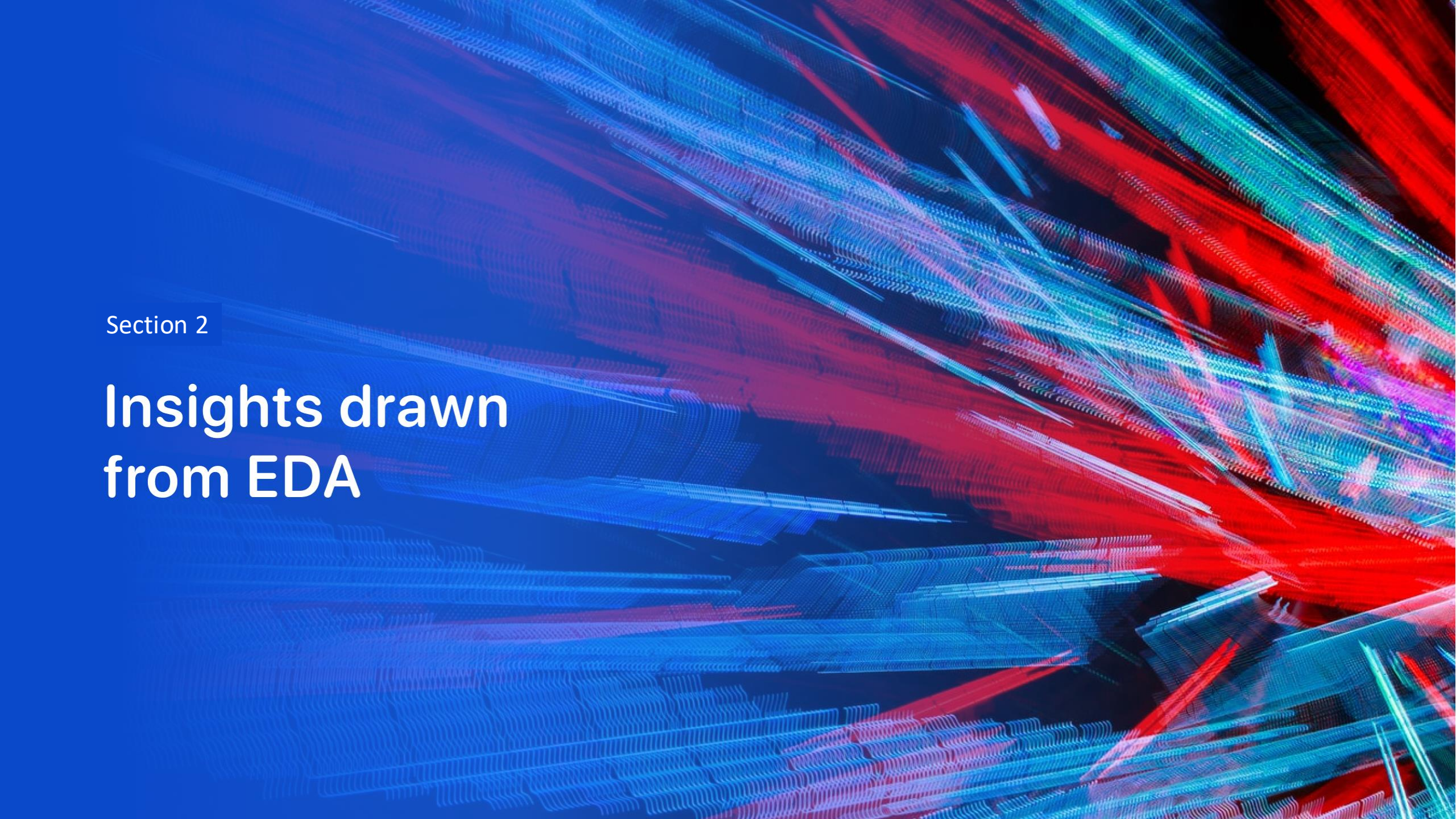
```
+-----+
|                                     |
|                               1. DATA PREPARATION                       |
| - Created target variable array from 'Class'                          |
| - Standardized feature matrix (StandardScaler)                       |
| - Divided dataset into Train/Test sets (80/20 split)                  |
+-----+
|                                     |
|                               v                                         |
+-----+
|                                     |
|                               2. PIPELINE DESIGN & TUNING                |
| - Initiated estimators: LogReg, SVM, Tree, KNN                       |
| - Defined hyperparameter tuning grids                                |
| - Structured 10-Fold Cross-Validation loops                           |
+-----+
|                                     |
|                               v                                         |
+-----+
|                                     |
|                               3. OPTIMIZATION & TRAINING                 |
| - Executed GridSearchCV for hyperparameter exploration               |
| - Isolated optimal model configurations                               |
| - Recorded highest training cross-validation scores                   |
+-----+
|                                     |
|                               v                                         |
+-----+
|                                     |
|                               4. EVALUATION & SELECTION                 |
| - Assessed generalization on unseen test data                        |
| - Compiled and compared accuracy metrics                             |
| - Selected best-performing landing prediction model                  |
+-----+
```

Results

Based on the evaluation of the classification models, here is the summary of the performance results:

Model Performance Summary

- **Identical Performance:** All four optimized models achieved the exact same accuracy of **83.33%** on the test dataset, correctly predicting 15 out of the 18 landing outcomes.
- **Confusion Matrix Insights:** For all models, the breakdown of the test predictions revealed:
- **True Negatives:** 3 failed landings were correctly identified.
- **False Positives:** 3 failed landings were incorrectly predicted as successful.
- **True Positives:** 12 successful landings were perfectly predicted.
- **False Negatives:** 0 successful landings were missed.
- **The Best Model:** Since all models generated identical predictive results on the unseen test data, they technically performed equally well. However, **Logistic Regression** or **SVM** are practically preferred for deployment due to their mathematical simplicity, stability, and lower computational overhead compared to instance-based methods like KNN.

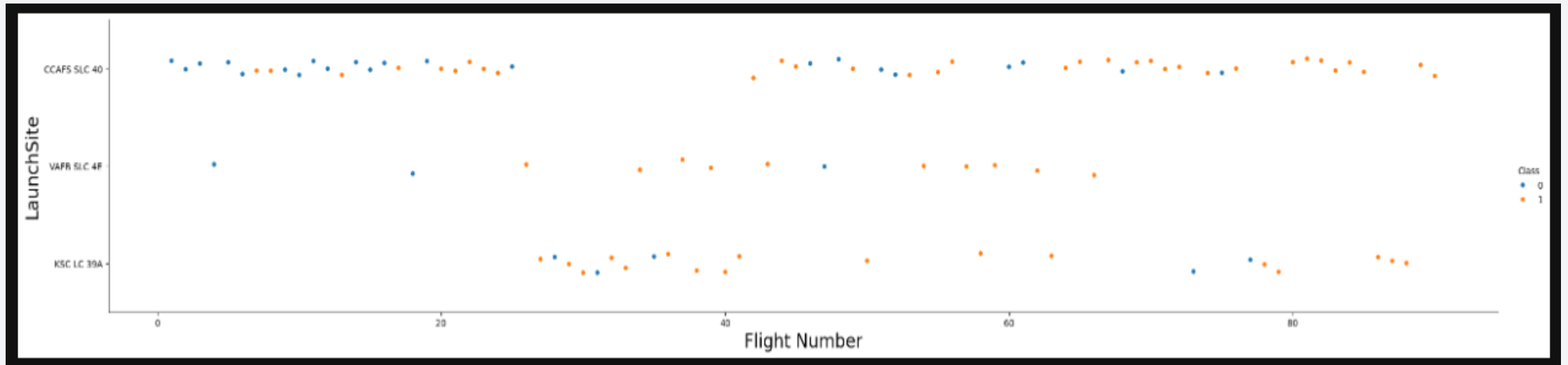
The background of the slide is an abstract composition. It features a dark blue base color. Overlaid on this are numerous diagonal streaks in shades of blue and red, creating a sense of motion or data flow. A faint, light blue grid pattern is also visible, particularly in the lower-left quadrant. The overall effect is high-tech and digital.

Section 2

Insights drawn from EDA

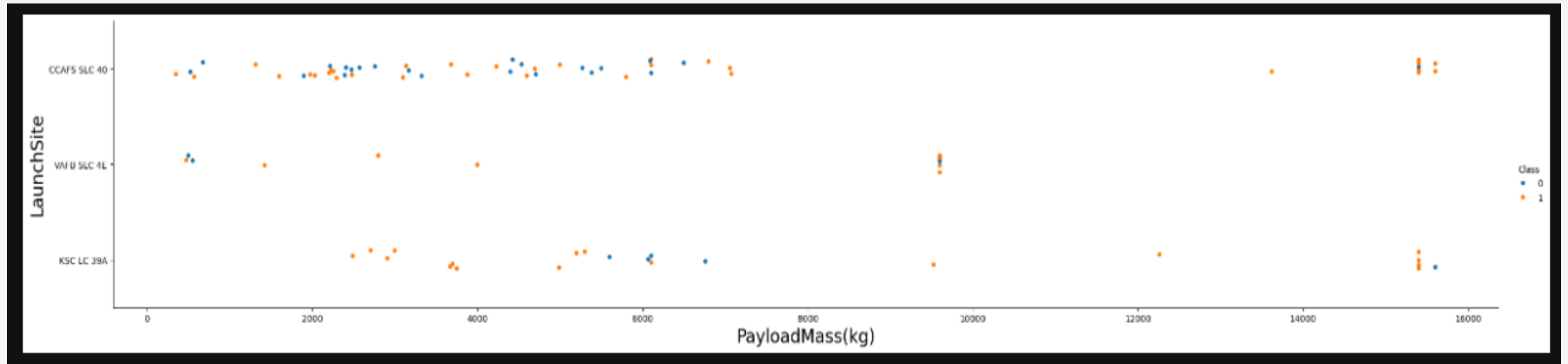
Flight Number vs. Launch Site

Launches from site CCAFS SLC 40 are the highest.



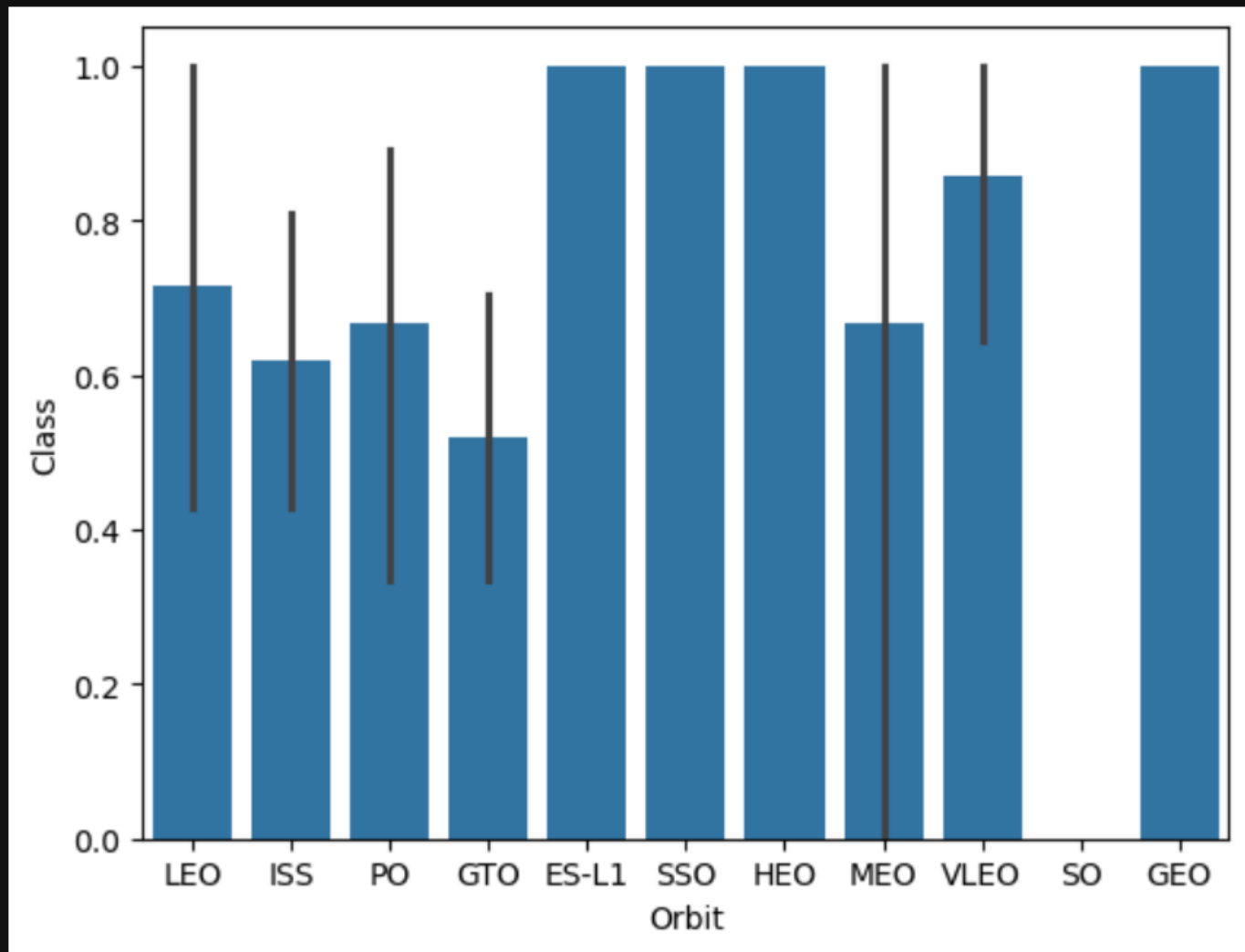
Payload vs. Launch Site

Payloads with lower mass have higher launches.



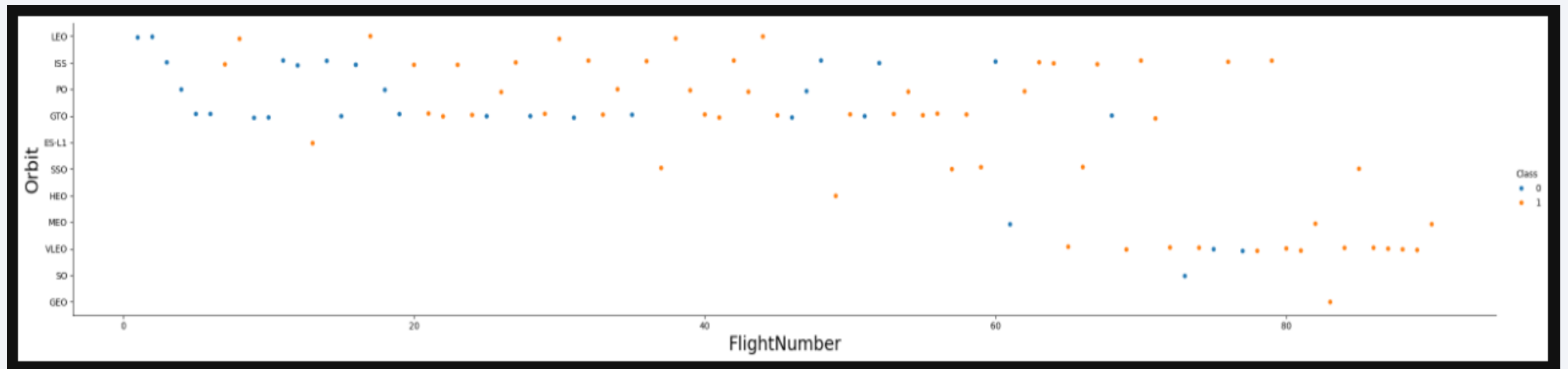
Success Rate vs. Orbit Type

```
[12]: <Axes: xlabel='Orbit', ylabel='Class'>
```



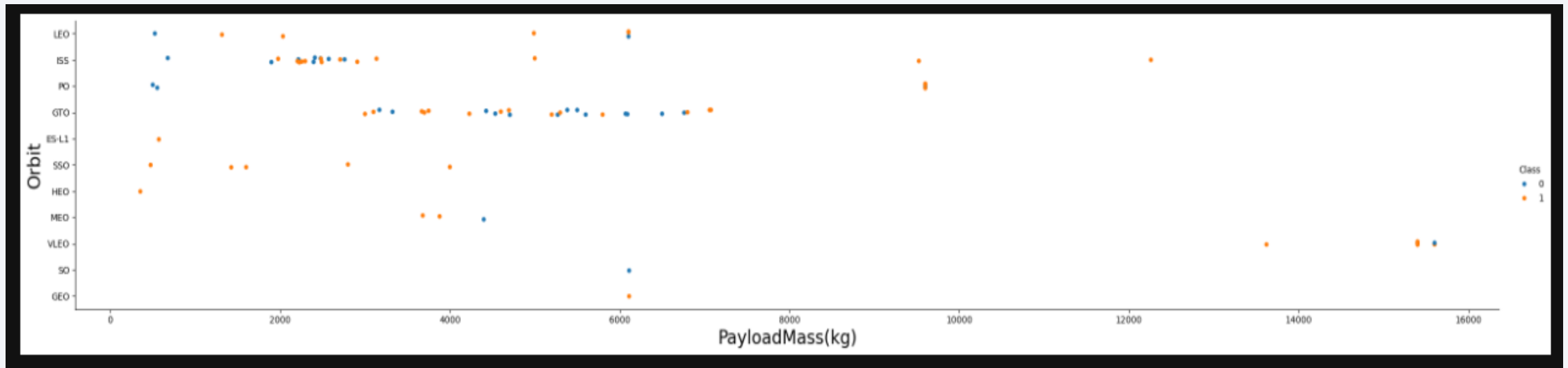
Flight Number vs. Orbit Type

More launches have shifted to VLEO in the recent years



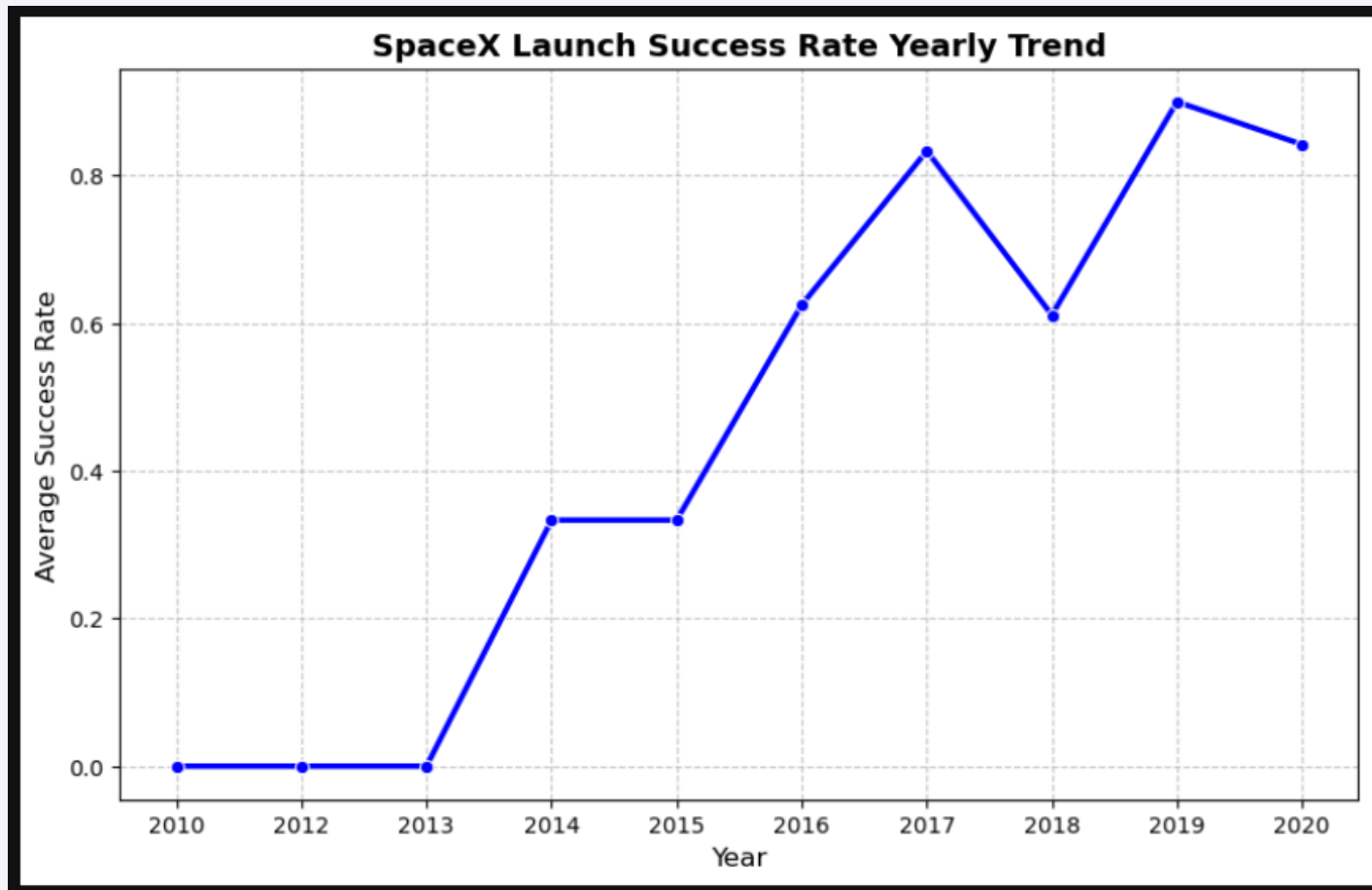
Payload vs. Orbit Type

With heavy payloads the successful landing or positive landing rate are more for Polar, LEO and ISS. With heavy payloads the successful landing or positive landing rate are more for Polar, LEO and ISS.



Launch Success Yearly Trend

The success rate since 2013 kept increasing till 2017 (stable in 2014) and after 2015 it started increasing.



All Launch Site Names

Display the names of the unique launch sites in the space mission

```
[23]: %%sql
      SELECT DISTINCT Launch_Site
      FROM SPACEXTBL;
```

```
* sqlite:///my_data1.db
Done.
```

```
[23]: Launch_Site
```

```
CCAFS LC-40
```

```
VAFB SLC-4E
```

```
KSC LC-39A
```

```
CCAFS SLC-40
```


Launch Site Names Begin with 'CCA'

[25]:

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Total Payload Mass

Total Payload Mass = 45596

```
[37]: %%sql
      SELECT Customer, SUM(PAYLOAD_MASS_KG_)
      FROM SPACEXTBL
      WHERE Customer = 'NASA (CRS)';
```

```
    * sqlite:///my_data1.db
Done.
```

```
[37]: Customer  SUM(PAYLOAD_MASS_KG_)
      -----
      NASA (CRS)                45596
```

Average Payload Mass by F9 v1.1

Average Payload Mass by F9 V1.1 is 2928.4

```
[38]: %%sql
SELECT Booster_Version, AVG(PAYLOAD_MASS__KG_)
FROM SPACEXTBL
WHERE Booster_Version = 'F9 v1.1';
```

```
* sqlite:///my_data1.db
Done.
```

Booster_Version	AVG(PAYLOAD_MASS_KG_)
F9 v1.1	2928.4

First Successful Ground Landing Date

The first successful ground landing was on 22nd December 2015

```
[53]: %%sql
      SELECT MIN(Date) AS FIRST_SUCCESS_LANDING_DATE
      FROM SPACEXTBL
      WHERE Landing_Outcome = 'Success (ground pad)';
```

```
    * sqlite:///my_data1.db
Done.
```

```
[53]: FIRST_SUCCESS_LANDING_DATE
      2015-12-22
```

Successful Drone Ship Landing with Payload between 4000 and 6000

```
[55]: %%sql
      SELECT Booster_Version
      FROM SPACEXTBL
      WHERE (Landing_Outcome = 'Success (drone ship)') AND PAYLOAD_MASS__KG_ BETWEEN 4000 AND 6000;
```

```
    * sqlite:///my_data1.db
Done.
```

```
[55]: Booster_Version
```

```
F9 FT B1022
```

```
F9 FT B1026
```

```
F9 FT B1021.2
```

```
F9 FT B1031.2
```

Total Number of Successful and Failure Mission Outcomes

```
[62]: %%sql
      SELECT Mission_Outcome, COUNT(Mission_Outcome)
      FROM SPACEXTBL
      GROUP BY Mission_Outcome;
```

```
    * sqlite:///my_data1.db
Done.
```

```
[62]:
```

Mission_Outcome	COUNT(Mission_Outcome)
Failure (in flight)	1
Success	98
Success	1
Success (payload status unclear)	1

Boosters Carried Maximum Payload

```
[67]: %%sql
      SELECT Booster_Version
      FROM SPACEXTBL
      WHERE PAYLOAD_MASS_KG_ = (SELECT MAX(PAYLOAD_MASS_KG_) FROM SPACEXTBL);

* sqlite:///my_data1.db
Done.
```

```
[67]: Booster_Version
```

F9 B5 B1048.4

F9 B5 B1049.4

F9 B5 B1051.3

F9 B5 B1056.4

F9 B5 B1048.5

F9 B5 B1051.4

F9 B5 B1049.5

F9 B5 B1060.2

F9 B5 B1058.3

F9 B5 B1051.6

F9 B5 B1060.3

F9 B5 B1049.7

2015 Launch Records

```
[68]: %%sql
SELECT
    CASE substr(Date, 6, 2)
        WHEN '01' THEN 'January' WHEN '02' THEN 'February' WHEN '03' THEN 'March'
        WHEN '04' THEN 'April'   WHEN '05' THEN 'May'       WHEN '06' THEN 'June'
        WHEN '07' THEN 'July'    WHEN '08' THEN 'August'    WHEN '09' THEN 'September'
        WHEN '10' THEN 'October' WHEN '11' THEN 'November' WHEN '12' THEN 'December'
    END AS Month_Name,
    Landing_Outcome,
    Booster_Version,
    Launch_Site
FROM SPACEXTBL
WHERE substr(Date, 1, 4) = '2015'
    AND Landing_Outcome = 'Failure (drone ship)';
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
[68]:
```

Month_Name	Landing_Outcome	Booster_Version	Launch_Site
January	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
April	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

```
[69]: %%sql
      SELECT
        Landing_Outcome,
        COUNT(Landing_Outcome) AS Total_Count
      FROM SPACEXTBL
      WHERE Date BETWEEN '2010-06-04' AND '2017-03-20'
      GROUP BY Landing_Outcome
      ORDER BY Total_Count DESC;
```

```
* sqlite:///my_data1.db
```

```
Done.
```

```
[69]:
```

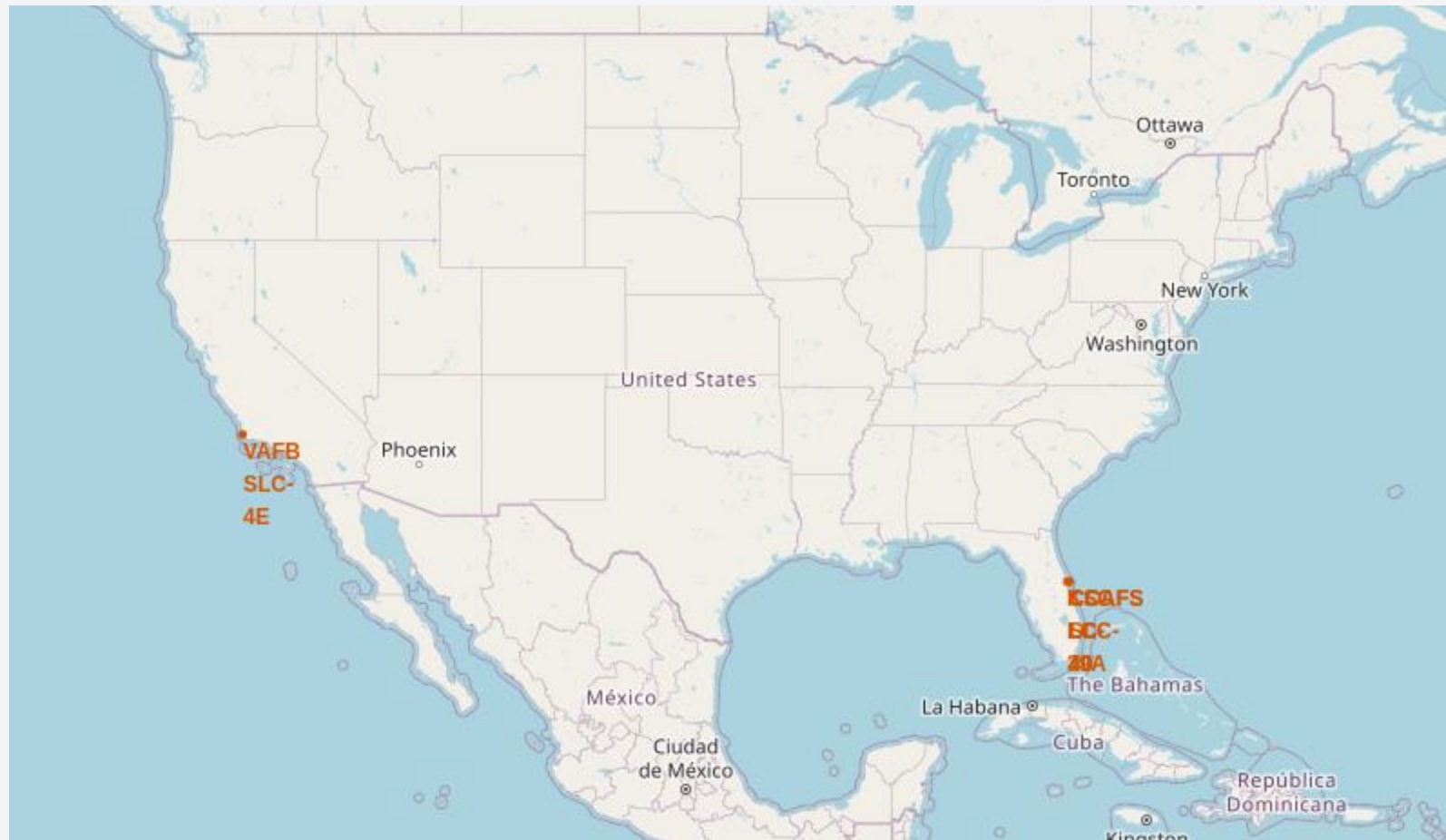
Landing_Outcome	Total_Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The image is a composite of a solid blue background on the left and a satellite photograph of Earth on the right. The Earth's surface is dark, with numerous bright yellow and orange lights representing cities and urban areas. The horizon of the Earth is visible as a curved line separating the dark surface from the deep blue of space.

Section 3

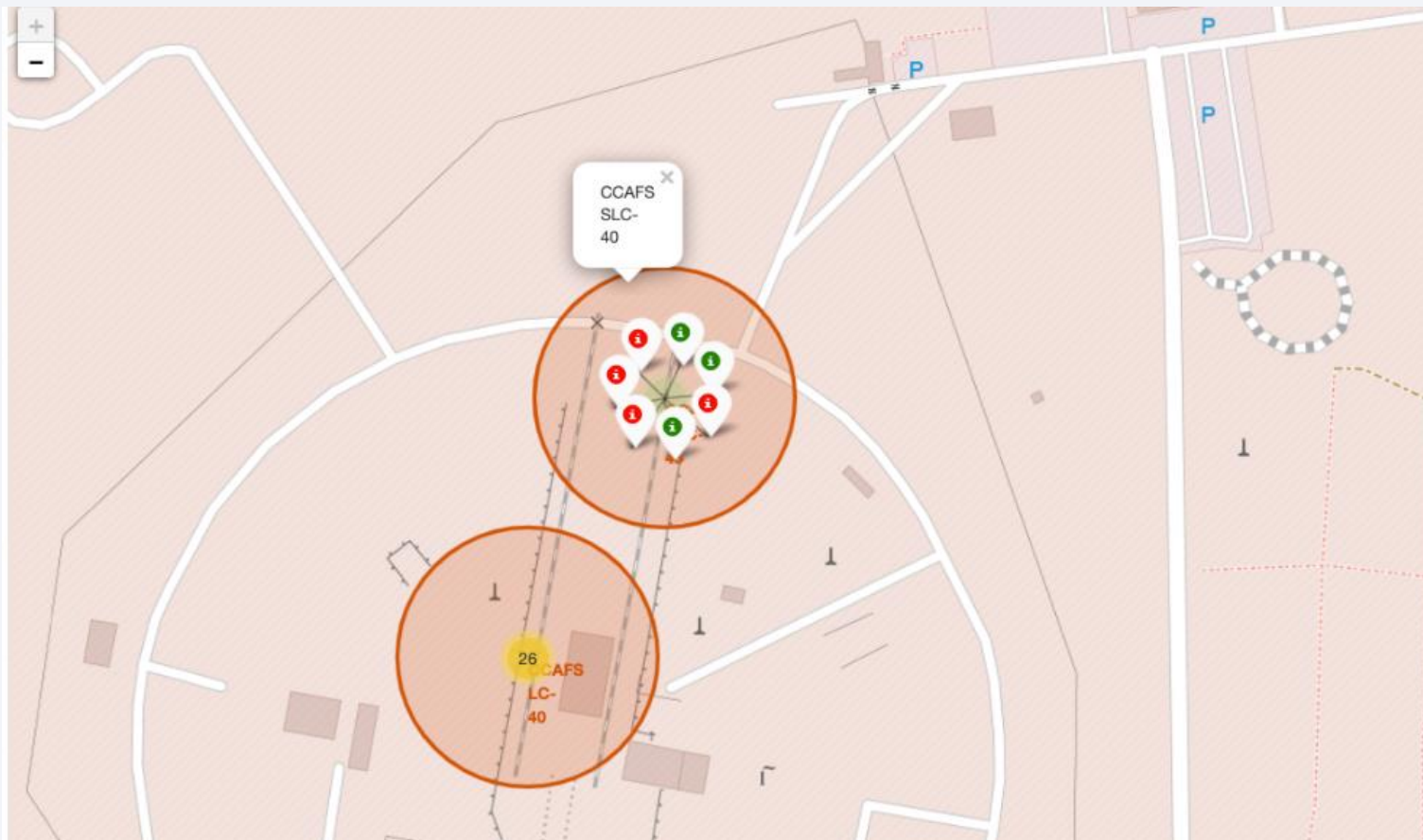
Launch Sites Proximities Analysis

All Launch sites on a map.

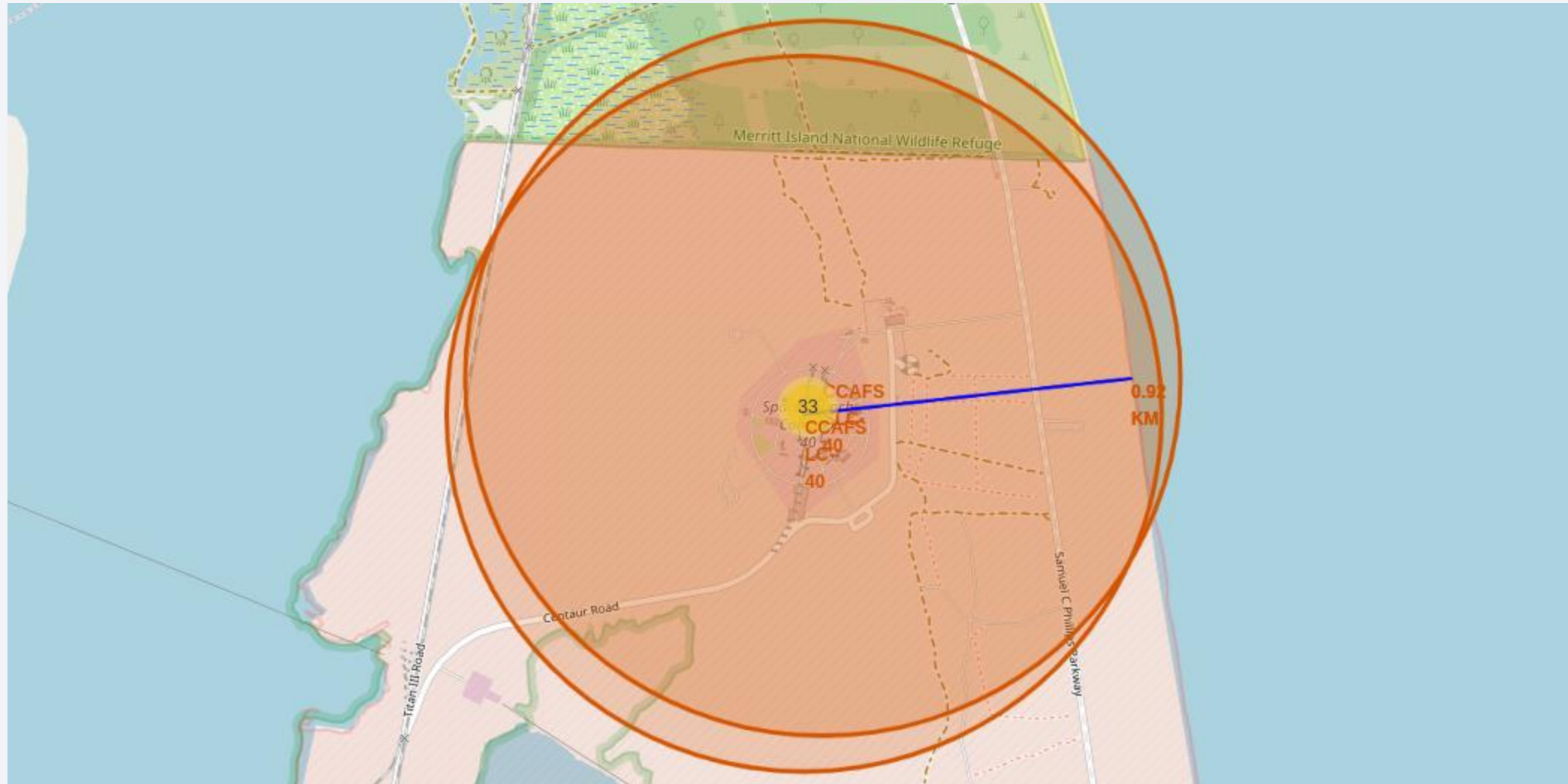


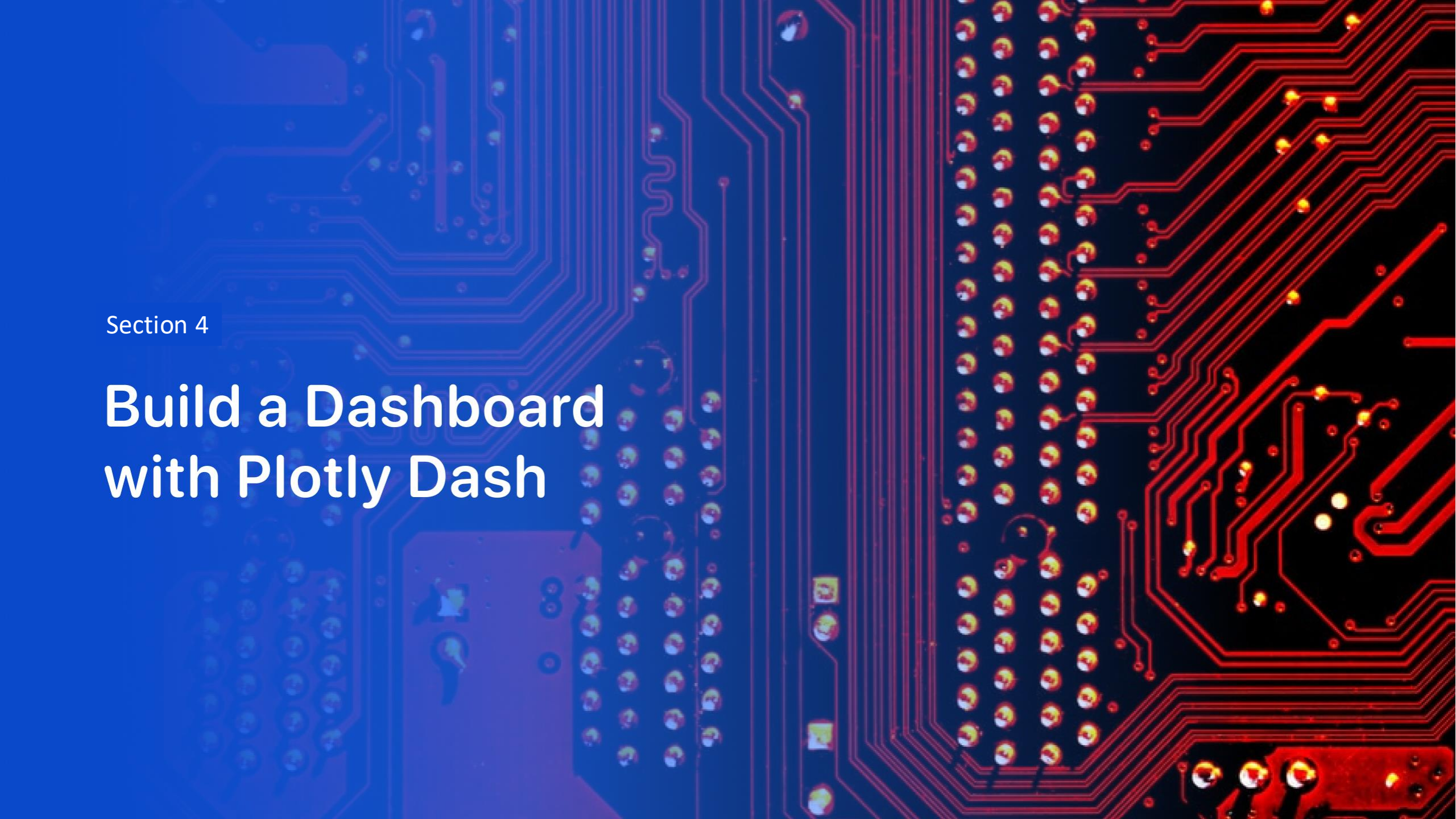
Successful/Failed Launches on the Map

Green markers represent successful launches while red markers represents failed launches.



Distance between Launch site to its proximities.

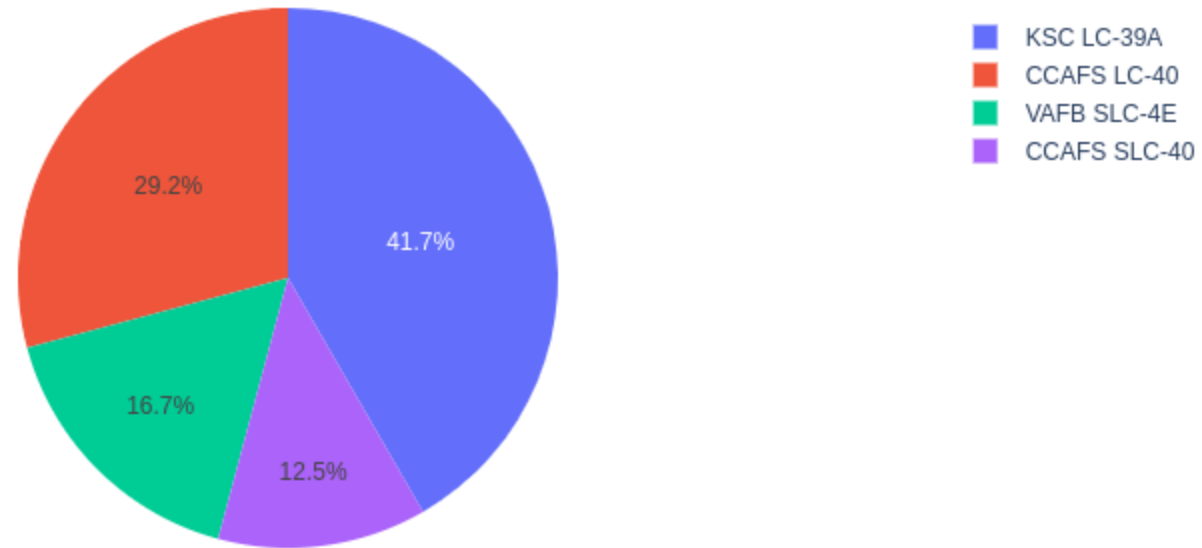




Section 4

Build a Dashboard with Plotly Dash

Launch Success Count for all sites

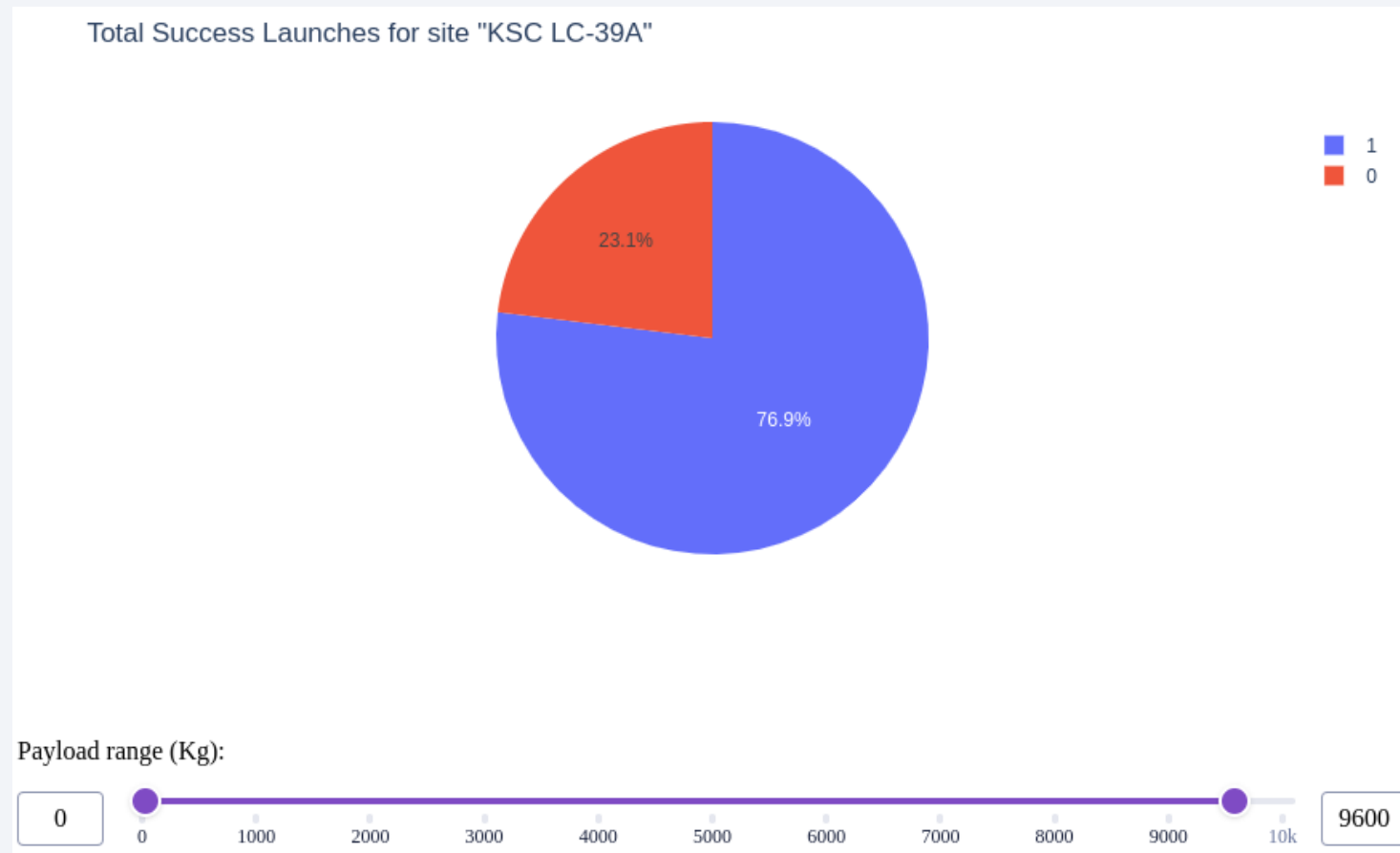


Payload range (Kg):



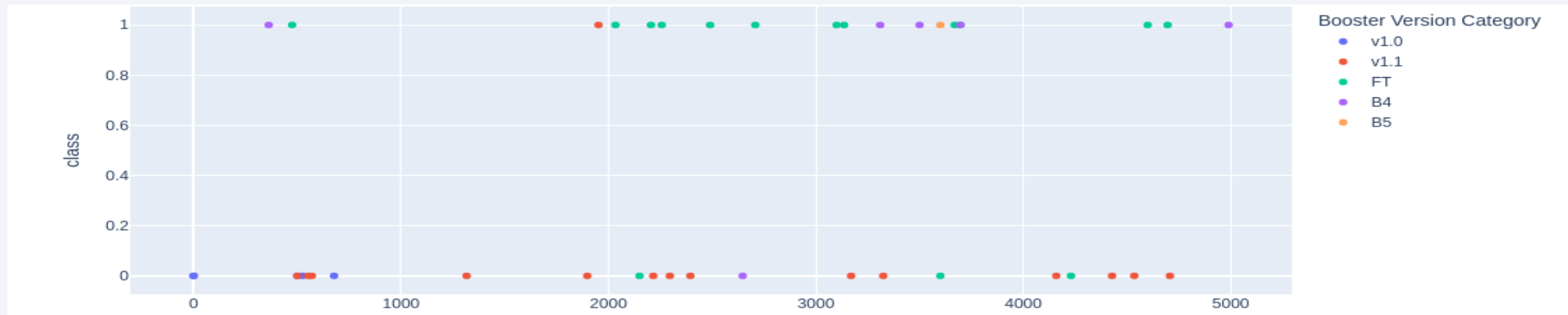
Launch Site with Highest Launch Success Rate

KCS LC 39A has the highest success rate of 76.9%

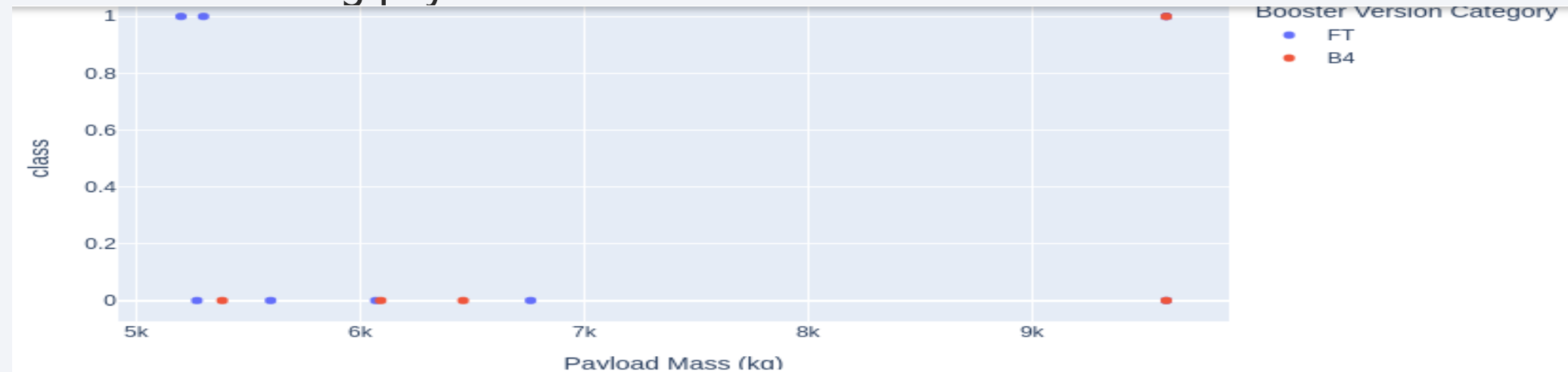


Payload vs. Launch Outcome

- 0-5000kg payload



- 5000-10000kg payload

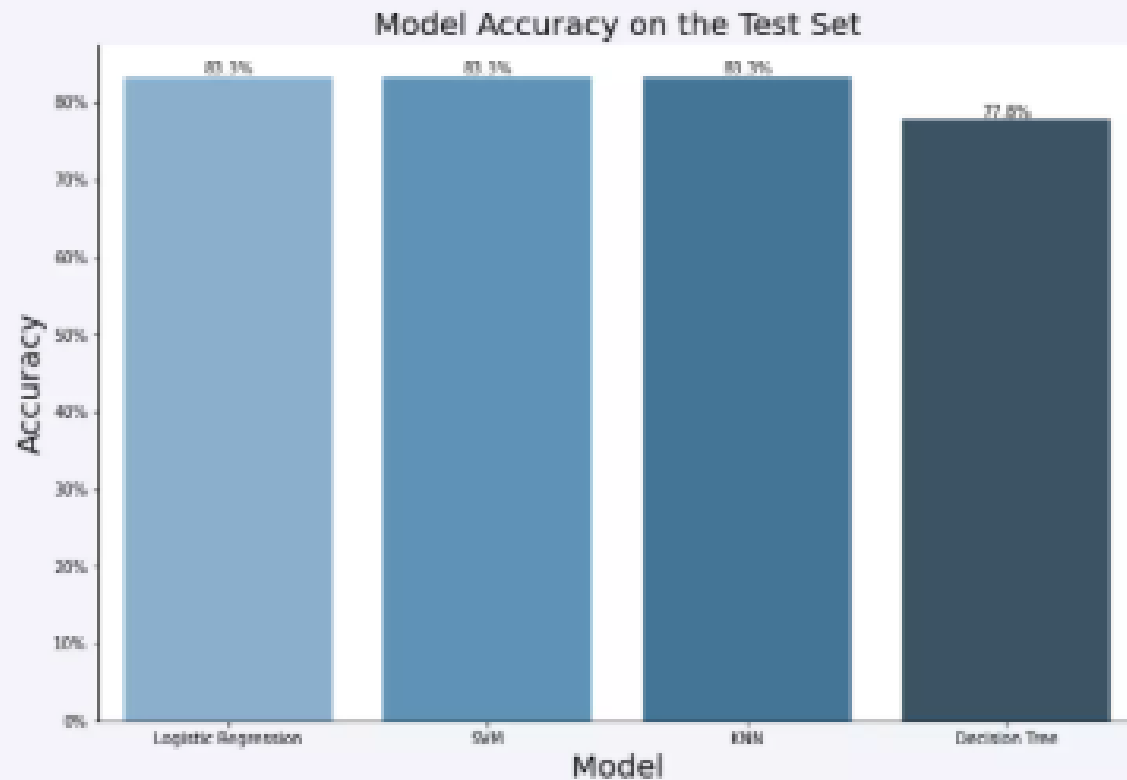


Section 5

Predictive Analysis (Classification)

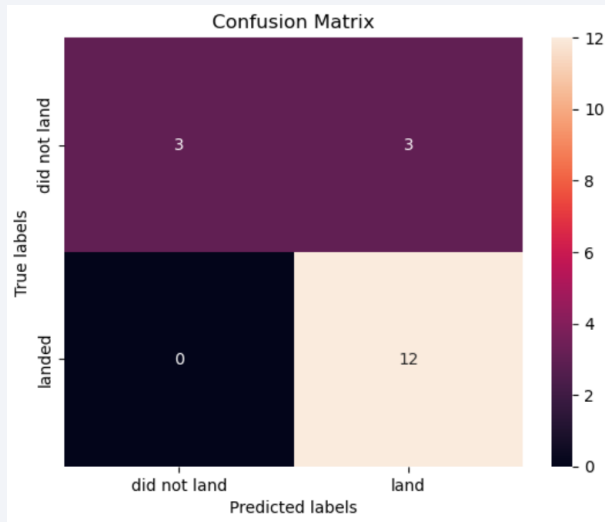
Classification Accuracy

The best performers included LR, SVM and KNN with an accuracy of 83.33%

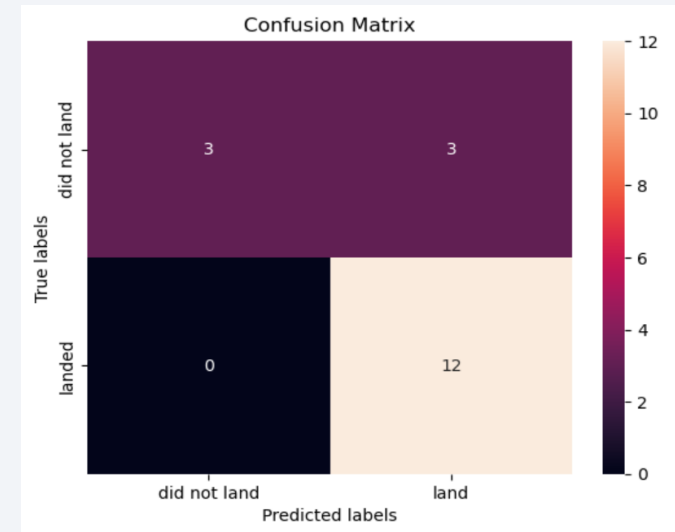


Confusion Matrix

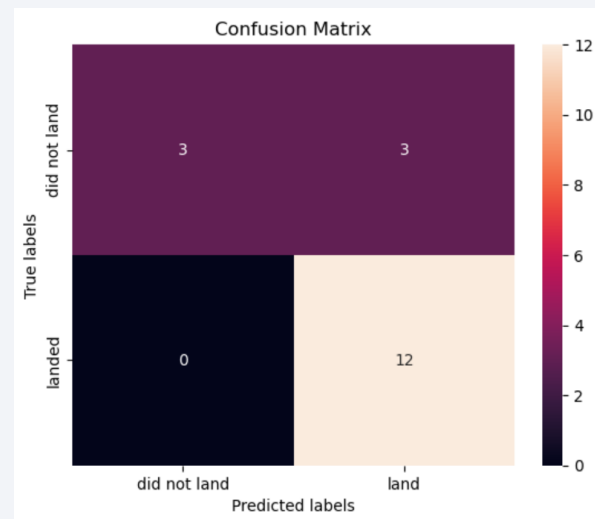
Logistic Regression



SVM



KNN



Conclusions

- **Increasing Success Over Time:** SpaceX's launch success rate has shown a clear upward trajectory since 2013, peaking in 2019/2020 at over 90% as the technology matured.
- **Impact of Orbit & Payload:** Higher landing success rates are strongly associated with specific orbits (like LEO, VLEO, and Polar), and launch profiles have significantly shifted toward VLEO in recent years.
- **Site-Specific Performance:** While CCAFS SLC-40 has hosted the highest volume of launches, KSC LC-39A boasts the highest overall launch success rate at 76.9%.
- **Robust Predictive Modeling:** Machine learning models reliably predict booster landing success with an identical test accuracy of 83.33% across Logistic Regression, SVM, and KNN. Logistic Regression or SVM are preferred for deployment due to their lower computational overhead.

Appendix

- **Data Sources & Pipelines:** Python scripts using requests for the SpaceX REST API and BeautifulSoup for parsing Wikipedia HTML tables.
- **Exploratory Data Analysis:** Comprehensive SQL queries covering payload aggregations, chronological firsts (e.g., first ground landing on 2015-12-22), and launch site outcome rankings.
- **Interactive Visualization Code:** Folium scripts for geographical proximity and launch site safety mapping; Plotly Dash application code for real-time payload vs. success rate dashboards.
- **Model Tuning Configurations:** Hyperparameter grids and 10-fold cross-validation loops used in GridSearchCV to optimize the classification pipelines.

Thank you!

